Structure and Interpretation *of* Computer Programs



SECOND EDITION

Unofficial Texinfo Format 2.andresraba5.6

Harold Abelson and
Gerald Jay Sussman
with Julie Sussman
foreword by Alan J. Perlis

©1996 by The Massachusetts Institute of Technology

Structure and Interpretation of Computer Programs,
second edition

Harold Abelson and Gerald Jay Sussman
with Julie Sussman, foreword by Alan J. Perlis



This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License
(CC BY-SA 4.0). Based on a work at mitpress.mit.edu.

The MIT Press
Cambridge, Massachusetts
London, England

McGraw-Hill Book Company
New York, St. Louis, San Francisco,
Montreal, Toronto

Unofficial Texinfo Format 2.andresraba5.6 (February 2, 2016),
based on 2.neilvandyke4 (January 10, 2007).

Contents

Unofficial Texinfo Format ix

Dedication xii

Foreword xiii

Preface to the Second Edition xix

Preface to the First Edition xxi

Acknowledgments xxv

1 Building Abstractions with Procedures 1

1.1 The Elements of Programming 6

1.1.1 Expressions 7

1.1.2 Naming and the Environment 10

1.1.3 Evaluating Combinations 12

1.1.4 Compound Procedures 15

1.1.5 The Substitution Model for Procedure Application 18

1.1.6 Conditional Expressions and Predicates 22

1.1.7 Example: Square Roots by Newton's Method . . 28

1.1.8 Procedures as Black-Box Abstractions 33
1.2 Procedures and the Processes They Generate 40
1.2.1 Linear Recursion and Iteration 41
1.2.2 Tree Recursion 47
1.2.3 Orders of Growth 54
1.2.4 Exponentiation 57
1.2.5 Greatest Common Divisors 62
1.2.6 Example: Testing for Primality 65
1.3 Formulating Abstractions with Higher-Order Procedures 74
1.3.1 Procedures as Arguments 76
1.3.2 Constructing Procedures Using `lambda` 83
1.3.3 Procedures as General Methods 89
1.3.4 Procedures as Returned Values 97

2 Building Abstractions with Data 107
2.1 Introduction to Data Abstraction 112
2.1.1 Example: Arithmetic Operations for Rational Numbers 113
2.1.2 Abstraction Barriers 118
2.1.3 What Is Meant by Data? 122
2.1.4 Extended Exercise: Interval Arithmetic 126
2.2 Hierarchical Data and the Closure Property 132
2.2.1 Representing Sequences 134
2.2.2 Hierarchical Structures 147
2.2.3 Sequences as Conventional Interfaces 154
2.2.4 Example: A Picture Language 172
2.3 Symbolic Data 192
2.3.1 Quotation 192

2.3.2 Example: Symbolic Differentiation 197
2.3.3 Example: Representing Sets 205
2.3.4 Example: Huffman Encoding Trees 218
2.4 Multiple Representations for Abstract Data 229
2.4.1 Representations for Complex Numbers 232
2.4.2 Tagged data 237
2.4.3 Data-Directed Programming and Additivity . . 242
2.5 Systems with Generic Operations 254
2.5.1 Generic Arithmetic Operations 255
2.5.2 Combining Data of Different Types 262
2.5.3 Example: Symbolic Algebra 274

3 Modularity, Objects, and State 294
3.1 Assignment and Local State 296
3.1.1 Local State Variables 297
3.1.2 The Benefits of Introducing Assignment 305
3.1.3 The Costs of Introducing Assignment 311
3.2 The Environment Model of Evaluation 320
3.2.1 The Rules for Evaluation 322
3.2.2 Applying Simple Procedures 327
3.2.3 Frames as the Repository of Local State 330
3.2.4 Internal Definitions 337
3.3 Modeling with Mutable Data 341
3.3.1 Mutable List Structure 342
3.3.2 Representing Queues 353
3.3.3 Representing Tables 360
3.3.4 A Simulator for Digital Circuits 369
3.3.5 Propagation of Constraints 386
3.4 Concurrency: Time Is of the Essence 401

3.4.1 The Nature of Time in Concurrent Systems . . 403
3.4.2 Mechanisms for Controlling Concurrency . . . 410
3.5 Streams . . . 428
3.5.1 Streams Are Delayed Lists . . . 430
3.5.2 Infinite Streams . . . 441
3.5.3 Exploiting the Stream Paradigm . . . 453
3.5.4 Streams and Delayed Evaluation . . . 470
3.5.5 Modularity of Functional Programs and Modularity of Objects . . . 479

4 Metalinguistic Abstraction 487
4.1 The Metacircular Evaluator . . . 492
4.1.1 The Core of the Evaluator . . . 495
4.1.2 Representing Expressions . . . 501
4.1.3 Evaluator Data Structures . . . 512
4.1.4 Running the Evaluator as a Program . . . 518
4.1.5 Data as Programs . . . 522
4.1.6 Internal Definitions . . . 526
4.1.7 Separating Syntactic Analysis from Execution . 534
4.2 Variations on a Scheme — Lazy Evaluation . . . 541
4.2.1 Normal Order and Applicative Order . . . 542
4.2.2 An Interpreter with Lazy Evaluation . . . 544
4.2.3 Streams as Lazy Lists . . . 555
4.3 Variations on a Scheme — Nondeterministic Computing 559
4.3.1 Amb and Search . . . 561
4.3.2 Examples of Nondeterministic Programs . . . 567
4.3.3 Implementing the amb Evaluator . . . 578
4.4 Logic Programming . . . 594
4.4.1 Deductive Information Retrieval . . . 599

4.4.2 How the Query System Works 615
4.4.3 Is Logic Programming Mathematical Logic? . . 627
4.4.4 Implementing the Query System 635
4.4.4.1 The Driver Loop and Instantiation . . 636
4.4.4.2 The Evaluator 638
4.4.4.3 Finding Assertions by Pattern Matching 642
4.4.4.4 Rules and Unification 645
4.4.4.5 Maintaining the Data Base 651
4.4.4.6 Stream Operations 654
4.4.4.7 Query Syntax Procedures 656
4.4.4.8 Frames and Bindings 659

5 Computing with Register Machines 666
5.1 Designing Register Machines 668
5.1.1 A Language for Describing Register Machines . 672
5.1.2 Abstraction in Machine Design 678
5.1.3 Subroutines 681
5.1.4 Using a Stack to Implement Recursion 686
5.1.5 Instruction Summary 695
5.2 A Register-Machine Simulator 696
5.2.1 The Machine Model 698
5.2.2 The Assembler 704
5.2.3 Generating Execution Procedures for Instructions 708
5.2.4 Monitoring Machine Performance 718
5.3 Storage Allocation and Garbage Collection 723
5.3.1 Memory as Vectors 724
5.3.2 Maintaining the Illusion of Infinite Memory . . 731

5.4 The Explicit-Control Evaluator . . . 741
5.4.1 The Core of the Explicit-Control Evaluator . . . 743
5.4.2 Sequence Evaluation and Tail Recursion . . . 751
5.4.3 Conditionals, Assignments, and Definitions . . 756
5.4.4 Running the Evaluator . . . 759
5.5 Compilation . . . 767
5.5.1 Structure of the Compiler . . . 772
5.5.2 Compiling Expressions . . . 779
5.5.3 Compiling Combinations . . . 788
5.5.4 Combining Instruction Sequences . . . 797
5.5.5 An Example of Compiled Code . . . 802
5.5.6 Lexical Addressing . . . 817
5.5.7 Interfacing Compiled Code to the Evaluator . . 823

References **834**

List of Exercises **844**

List of Figures **846**

Index **848**

Colophon **855**